

Ridge Regression with Gradient Descent

Contents

1. Overview	2
2. The gradient	2
2.1 The loss with a factor of $\frac{1}{2}$	2
2.2 Expanding and differentiating	2
2.3 The update rule	3
3. Seeing the paths	3
4. Code from scratch	4
5. Gradient descent stops early	5
6. Ridge with gradient descent in scikit-learn	6
7. Summary	7
8. Key terms	7

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Standardization ([Note 24](#)); Multiple linear regression ([Note 55](#)); Normal equation ([Note 55](#)); Gradient descent ([Note 57](#)); Bias-variance trade-off ([Note 62](#)); Regularisation ([Note 63](#)).
- **Leads to:** Elastic Net ([Note 69](#)).
- **Compare with:** Lasso regression ([Note 67](#)).

1. Overview

Key point: Ridge can also be trained with gradient descent. The gradient is the linear regression gradient plus λw , so each step also pulls the coefficients a little towards 0.

The previous Note found the Ridge coefficients in one step with a formula. Like plain linear regression, Ridge can instead be trained with **gradient descent**: start somewhere, and repeatedly step downhill on the loss.

Gradient descent is useful when there are many input columns, because the formula needs the inverse of a large matrix (the gradient descent Notes). This Note derives the Ridge gradient, codes it from scratch, and shows the scikit-learn options.

2. The gradient

Key point: With a factor of $\frac{1}{2}$ in the loss, the gradient is $X^T X w - X^T y + \lambda w$.

2.1 The loss with a factor of $\frac{1}{2}$

Key point: Multiplying the loss by $\frac{1}{2}$ does not move its minimum; it only removes a 2 from the derivative.

The Ridge loss from the previous Note, multiplied by $\frac{1}{2}$:

$$L = \frac{1}{2}(Xw - y)^T(Xw - y) + \frac{1}{2}\lambda w^T w$$

Halving every value of the loss leaves the lowest point at the same w . It just makes the derivative tidier, as the 2 from differentiating a square cancels the $\frac{1}{2}$.

2.2 Expanding and differentiating

Key point: The same expansion as the previous Note, then the same derivative rules.

Expanding the product as in the normal equation Note:

$$L = \frac{1}{2} \left(w^\top X^\top X w - 2w^\top X^\top y + y^\top y \right) + \frac{1}{2} \lambda w^\top w$$

Differentiating with respect to w :

$$\frac{\partial L}{\partial w} = X^\top X w - X^\top y + \lambda w$$

The first two terms are the linear regression gradient. The Ridge penalty adds only λw .

Extra: Setting this gradient to zero gives $(X^\top X + \lambda I)w = X^\top y$, the formula of the previous Note. Gradient descent walks towards the same answer step by step instead of jumping to it.

2.3 The update rule

Key point: New w = old w minus the learning rate times the gradient.

As in every gradient descent Note, all coefficients are updated together:

$$w_{\text{new}} = w_{\text{old}} - \eta \left(X^\top X w_{\text{old}} - X^\top y + \lambda w_{\text{old}} \right)$$

where η is the learning rate. The first entry of w is the intercept, which is not penalised. So, as before, the λw term uses $\lambda I w$ with the top-left entry of I set to 0.

Extra: Rearranging the update gives $w_{\text{new}} = (1 - \eta\lambda) w_{\text{old}} - \eta (X^\top X w_{\text{old}} - X^\top y)$. So every step first shrinks the coefficients by the factor $1 - \eta\lambda$, then takes the ordinary linear regression step. This is why the L2 penalty is also called **weight decay**, the name used in deep learning.

3. Seeing the paths

Key point: Gradient descent finds the bottom of whichever loss it is given. With $\lambda = 100$, the bottom sits at a smaller slope.

Figure 1 runs 60 steps of gradient descent on the one-input example of the previous Notes, from the same start point ($m = -5$, $b = 20$, learning rate 0.005).

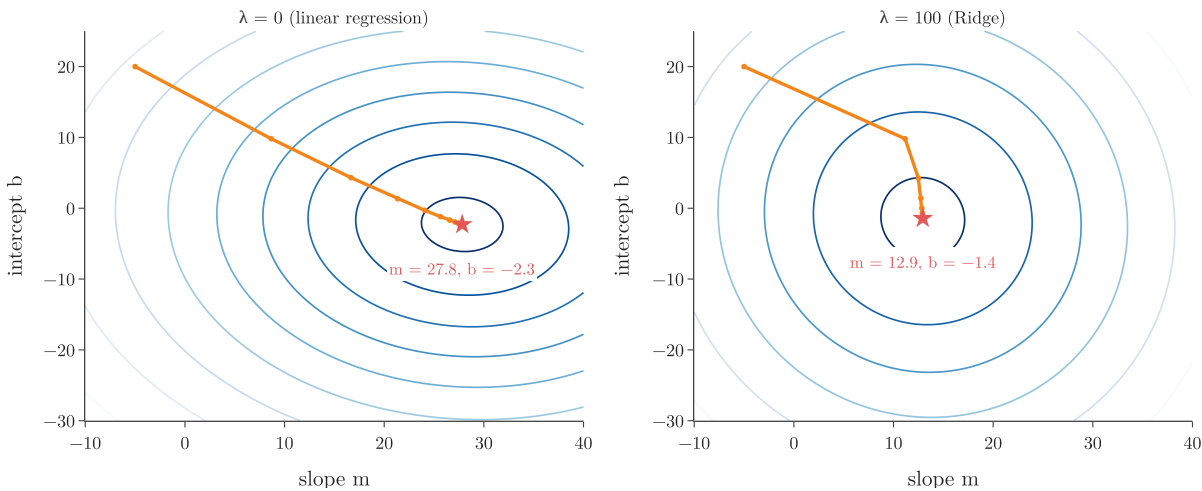


Figure 1: Gradient descent on the loss with $\lambda = 0$ and $\lambda = 100$

- $\lambda = 0$: the path ends at $m = 27.8$, $b = -2.3$, the linear regression answer.
- $\lambda = 100$: the penalty moves the bottom of the bowl to $m = 12.9$, $b = -1.4$. These are exactly the Ridge values from the formula in the previous Note.

The penalty also makes the bowl rounder, so the path heads more directly to the bottom.

4. Code from scratch

Key point: The loop computes the gradient with matrix products and updates all coefficients at once.

Python: Ridge with batch gradient descent.

```
class MyRidgeGD:
    def __init__(self, epochs, learning_rate, alpha):
        self.epochs, self.lr, self.alpha = (epochs,
                                             learning_rate, alpha)

    def fit(self, X, y):
        X = np.insert(X, 0, 1, axis=1)  # column of 1s
        w = np.insert(np.ones(X.shape[1] - 1), 0, 0.0)
        I = np.identity(X.shape[1])
        I[0, 0] = 0  # no penalty on intercept
        for _ in range(self.epochs):
            grad = X.T @ X @ w - X.T @ y + self.alpha * I @ w
            w = w - self.lr * grad
        self.intercept_, self.coef_ = w[0], w[1:]
        return self
```

```
def predict(self, X):
    return X @ self.coef_ + self.intercept_
```

On the diabetes data (test size 0.2, random state 4) with alpha 0.001, learning rate 0.005 and 500 epochs, test R^2 is 0.474 and the intercept 150.87.

Extra: This gradient adds up the errors of all rows instead of averaging them, so its size grows with the number of rows. That is why the learning rate must be tiny here. With 0.006 instead of 0.005, the steps overshoot and grow: after 100 epochs the largest coefficient is about 15,000.

5. Gradient descent stops early

Key point: After 500 epochs, gradient descent is still far from the exact answer. Its test R^2 is slightly higher only because stopping early also keeps coefficients small.

The exact Ridge answer from the previous Note scores test R^2 0.463 here. Figure 2 tracks gradient descent for up to a million epochs.

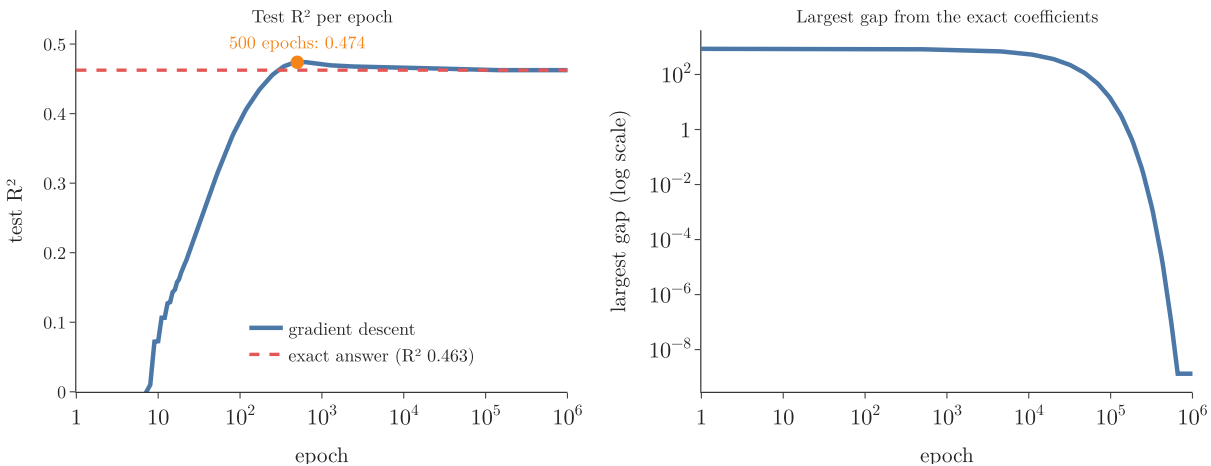


Figure 2: Test R^2 and distance from the exact answer, per epoch

Epochs	Test R^2	Largest gap from the exact coefficients
100	0.390	897
500	0.474	823
50,000	0.464	104
1,000,000	0.463	0.0

- **Left:** test R^2 rises, peaks around 500 epochs at 0.474, then drifts down to the exact answer's 0.463.

- **Right:** the coefficients need about a million epochs to reach the exact values.

The diabetes inputs are strongly correlated (the s1 and s5 pair of the previous Notes). Along such directions the bowl is almost flat, so gradient descent moves very slowly there. Stopping early leaves those coefficients small, which acts like extra regularisation and here happened to help.

Extra: Stopping training before the loss is fully minimised, chosen by watching a validation score, is a regularisation technique of its own called **early stopping**. It is widely used for neural networks.

6. Ridge with gradient descent in scikit-learn

Key point: Use `SGDRegressor` with `penalty="l2"`, or `Ridge` with an iterative solver.

Python: Two scikit-learn options.

```
from sklearn.linear_model import SGDRegressor, Ridge

sgd = SGDRegressor(penalty="l2", alpha=0.001, max_iter=500,
                   eta0=0.1, learning_rate="constant",
                   random_state=0)
sgd.fit(X_train, y_train)
sgd.score(X_test, y_test)           # R2 0.450

ridge = Ridge(alpha=0.001, max_iter=500, solver="sparse_cg")
ridge.fit(X_train, y_train)
ridge.score(X_test, y_test)         # R2 0.463
```

- **SGDRegressor** (the stochastic gradient descent Note) adds the Ridge penalty with `penalty="l2"`. Its `alpha` is the penalty strength.
- **Ridge** has a `solver` setting. `"cholesky"` and `"svd"` use the formula directly. `"sparse_cg"`, `"lsqr"`, `"sag"` and `"saga"` reach the answer step by step, and `max_iter` limits the steps.

Ridge with an iterative solver has no learning rate to set: these solvers choose their own step sizes. Here it reaches the exact answer, 0.463.

Extra: `SGDRegressor` averages the loss over the rows, while `Ridge` adds it up. So the same `alpha` means a stronger penalty in `SGDRegressor`: its `alpha` equals `Ridge`'s `alpha` divided by the number of rows. With 353 training rows, `SGDRegressor(alpha=0.001)` matches `Ridge(alpha=0.353)`.

7. Summary

Method	How it finds w	Test R^2 here
Formula (<code>Ridge</code> , cholesky)	one step, $(X^T X + \lambda I)^{-1} X^T y$	0.463
<code>Ridge</code> , <code>sparse_cg</code>	iterative solver	0.463
<code>MyRidgeGD</code> , 500 epochs	batch gradient descent, stopped early	0.474
<code>SGDRegressor</code> , l2	stochastic gradient descent	0.450

- The Ridge gradient is $X^T X w - X^T y + \lambda w$: the linear regression gradient plus λw .
- Each update shrinks the coefficients by $1 - \eta \lambda$ first, hence the name weight decay.
- The intercept is left out of the penalty.
- In scikit-learn: `SGDRegressor(penalty="l2")` or `Ridge` with an iterative solver.

8. Key terms

Term	Meaning
Weight decay	Another name for the L2 penalty: each gradient step shrinks the coefficients by a fixed factor
Early stopping	Stopping training before the loss is fully minimised, which keeps coefficients small
Solver	The method a scikit-learn model uses to find its coefficients
penalty	<code>SGDRegressor</code> setting that adds a regularisation penalty, such as “l2” for <code>Ridge</code>